P3669R0
Non-Blocking Support for `std::execution`

Published Proposal, 2025-04-14

This version:
http://wg21.link/P3669R0

Author:
Detlef Vollmann

Audience:
SG1, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

`std::execution` as currently specified doesn't provide support for non-blocking operations. This proposal tries to fix this.

Table of Contents

4.2 Modifications for Concurrent Queues

References

Informative References

§ 1. Revision History

This paper is the initial revision.

§ 2. Introduction

Some execution environments want to make sure that specific operations are non-blocking. These can not signal an event using the facilities of `std::execution` as currently specified as it doesn't provide operations that are guaranteed to be non-blocking.

This became apparent in the proposal for concurrent queues *C++ Concurrent Queues (P0260)*, which has support for non-blocking usage. But the non-blocking operations got a very weird interface when interfacing with `std::execution` due to the lack of non-blocking signalling to an async waiter.

In `std::execution` the basic operation to signal an event is to schedule a continuation. In practice this generally means that a `start` operation is called on an operation state that comes from a sender provided by a scheduler.

This proposal is independent from *C++ Concurrent Queues (P0260)* as the problem of signalling an event in a non-blocking way is not specific to concurrent queues.

§ 3. Design

This paper proposes to add a `bool try_start()` operation to the operation states that come from a scheduler and require it to be non-blocking (and to return `false` if it would block).

Unfortunately it is not possible for all kind of schedulers to provide such an opertion. Schedulers that are not backed by by an execution context that maintains a work queue but run the operation right away are an example. Schedulers that enqueue the work on a different system are another example.

For this reason, this paper proposes a new concept `concurrent_scheduler` derived from the `scheduler` concept. `concurrent_scheduler` requires the `try_start` operation.

While this proposal is generally independent from *C++ Concurrent Queues (P0260)*, we still would like to fix the weird interface in that proposal if possible. For this the proposed wording contains a part that modifies the wording for the queue concepts. The idea here is to make it ill-formed if an async operation is called for a queue that also implements `concurrent-queue` and its scheduler does not implement `concurrent_scheduler`.

4. Proposed Wording

(Sorry, the wording is formally horribly incomplete, but I think substantially it's fairly complete.)

4.1. Concurrency Concepts

4.1.1. Exposition-only Concurrent Op State Concept

1. The exposition-only concept *concurrent-op-state* defines the requirements of an operation state type ([async.ops]) that provides an operation to potentially start the operation without blocking ([defns.block]).

```
namespace std::execution {
  template <class O>
    concept concurrent-op-state =
      operation_state<O> &&
      requires (O& o) {
        { o.try_start() } noexcept -> bool
      };
}
```

2. In the following description, `O` denotes a type modeling the *concurrent-op-state* concept and `o` denotes an object of type `O`.
3. The expression `o.try_start()` has the following semantics:
    1. *Effects:* Starts ([async.ops]) the asynchronous operation associated with `o`, if it can be achieved without blocking.
    2. *Returns:* `true` if the async operation was started, `false` otherwise.

4.1.2. Concurrent Scheduler Concept

1. The concept `concurrent_scheduler` defines the requirements of a scheduler type ([async.ops]) that can provide a *concurrent-op-state*.

```
namespace std::execution {
  template<class S, receiver R>
    concept concurrent_scheduler =
      scheduler<S> &&
      requires (S&& s, R&& r) {
        { connect(schedule(std::forward<S>(s)), std::move(r) }
          -> concurrent-op-state;
    }
}
```

4.1.3. `run_loop`

1. *run-loop-scheduler* models `concurrent_scheduler`.

4.2. Modifications for Concurrent Queues

Remove `conqueue_errc::busy_async` from the enumeration and the respective return statements from `try_emplace`, `try_push` and `try_pop` in the *concurrent-queue* concept.

In the *async-concurrent-queue* concept add another bullet to the senders returned by `async_emplace` and `async_pop` after bullet 6:

7. If the scheduler of `r` does not model `concurrent_scheduler`, the program is ill-formed.

References

Informative References

[P0260]
C++ Concurrent Queues (P0260). URL: https://wg21.link/P0260